Reti logiche

Appunti

Marco Cerbella

Università degli Studi di Perugia – A.A. 2026/2027
Aggiornato al 19 ottobre 2026

Indice

1 Algebra di Boole e algebra di commutazione (Lezione 1 – 21 settembre 2026) 2

2 Funzioni ed espressioni booleane (Lezione 2 – 6 ottobre 2026) 4

3 Porte logiche e circuiti logici (Lezione 3) 10

4 Progettazione di reti logiche combinatorie (Lezione 4) 13

5 Ottimizzazione: mappe di Karnaugh (Lezione 5) 15

1 Algebra di Boole e algebra di commutazione

Lezione 1 – 21 settembre 2026

Cenno storico

L'algebra di Boole fu introdotta da George Boole (1815–1864) nell'Ottocento per dimostrare affermazioni logiche svolgendo calcoli algebrici: per Boole la logica va associata alla matematica e non alla filosofia. Nel 1936 Claude Shannon (1916–2001), studiando al MIT un circuito di oltre cento relè collegato all'analizzatore differenziale di Bush, intuì che l'algebra di Boole è un modello formale per l'*analisi* e la *sintesi* di tali sistemi: due interruttori in *serie* (*parallelo*) si descrivono con AND (OR), mentre il NOT si realizza con il contatto posteriore di un relè. Dalla sua tesi (1938) nasce l'algebra di commutazione, base delle reti logiche.

1.1 Algebra di Boole

Definizione. Un'*algebra di Boole* è una struttura algebrica

$$\mathcal{B} = \langle A, +, \cdot, ', 0, 1 \rangle$$

dove:

- A è l'insieme degli elementi su cui l'algebra opera (*supporto*);
- $+$ è l'operatore di disgiunzione (OR);
- $\cdot$ è l'operatore di congiunzione (AND);
- $'$ è l'operatore unario di complementazione (NOT);
- 0 è l'elemento neutro rispetto a $+$, 1 è l'elemento neutro rispetto a $\cdot$;

e valgono i seguenti assiomi (*postulati di Huntington*).

1.1.1 Postulati di Huntington

1. **Commutatività**: $\forall a, b \in A$: $a + b = b + a$ e $a \cdot b = b \cdot a$.
2. **Elementi neutri**: esistono 0 e 1 tali che, per ogni $a \in A$, $a + 0 = a$ e $a \cdot 1 = a$.
3. **Distributività** (di ciascuna operazione rispetto all'altra): $\forall a, b, c \in A$

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$
$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

4. **Complemento**: per ogni $a \in A$ esiste ed è unico a' tale che $a + a' = 1$ e $a \cdot a' = 0$.

Teorema (Principio di dualità). Ogni proposizione derivata dagli assiomi di Huntington resta valida scambiando ovunque tra loro le operazioni $+$ e $\cdot$ e gli elementi neutri 0 e 1.

Osservazione. In particolare se $\langle A, +, \cdot, ', 0, 1 \rangle$ è un'algebra di Boole, lo è anche $\langle A, \cdot, +, ', 1, 0 \rangle$.

1.1.2 Esempi

Esempio (Algebra delle classi)**.** Sia A un insieme e $\mathcal{P}(A)$ l'insieme delle sue parti (tutti i sottoinsiemi di A). La struttura $\langle \mathcal{P}(A), \cup, \cap, ^-, \emptyset, A\rangle$ è un'algebra di Boole:

- $\cup$ e $\cap$ sono commutative;
- $\emptyset$ è neutro rispetto a $\cup$ e A rispetto a $\cap$: per ogni $X \in \mathcal{P}(A)$, $X \cup \emptyset = X$ e $X \cap A = X$;
- distributività e unicità del complemento si verificano con i diagrammi di Venn.

È un'algebra di Boole *chiusa*: $\forall X, Y \in \mathcal{P}(A)$, $X \cap Y$, $X \cup Y$ e $\overline{X} = A \setminus X$ sono sottoinsiemi di A, quindi elementi di $\mathcal{P}(A)$.

Teorema (Rappresentazione di Stone). Per ogni algebra di Boole finita $\mathcal{A}$ esiste un insieme finito X tale che $\mathcal{A}$ è isomorfa a $\langle \mathcal{P}(X), \cup, \cap, ^-, \emptyset, X\rangle$.

Conseguenze:

- la cardinalità del supporto di un'algebra di Boole finita è sempre una potenza di 2;
- per dimostrare che le algebre di Boole finite godono di una proprietà basta dimostrarla per l'algebra delle classi (ad esempio con i diagrammi di Venn).

Esempio (Altre algebre di Boole)**.** $\langle \{0,1\}, \min, \max, 1-x, 0, 1\rangle$ è un'algebra di Boole; così come quella ottenuta scambiando le operazioni come nell'osservazione sopra.

1.1.3 Variabili booleane

Sia $\mathcal{A} = \langle A, +, \cdot, ', 0, 1\rangle$ un'algebra di Boole. Una *variabile booleana* è un simbolo che indica un qualsiasi elemento del supporto A; il suo valore è un elemento di A. Le variabili si indicano con le ultime lettere dell'alfabeto $(x, y, \dots)$, gli elementi del supporto con le prime $(a, b, \dots)$ e con i simboli 0 e 1.

1.1.4 Proprietà dell'algebra di Boole

Proprietà. Per ogni x, y, z valgono:

- **associatività**: $x + (y + z) = (x + y) + z$ e $x \cdot (y \cdot z) = (x \cdot y) \cdot z$;
- **idempotenza**: $x + x = x$ e $x \cdot x = x$;
- **elemento nullo**: $x + 1 = 1$ e $x \cdot 0 = 0$;
- **unicità del complemento**: x' è unico;
- **assorbimento**: $x + x \cdot y = x$ e $x \cdot (x + y) = x$;
- **semplificazione**: $x + x' \cdot y = x + y$ e $x \cdot (x' + y) = x \cdot y$;
- **involuzione**: $(x')' = x$;
- **leggi di De Morgan**: $(x + y)' = x' \cdot y'$ e $(x \cdot y)' = x' + y'$;
- **consenso**: $x \cdot y + x' \cdot z + y \cdot z = x \cdot y + x' \cdot z$ e $(x+y) \cdot (x'+z) \cdot (y+z) = (x+y) \cdot (x'+z)$.

Ciascuna si può verificare con i diagrammi di Venn, oppure dimostrare dagli assiomi.

Esempio (Dimostrazione dell'idempotenza)**.**

$$\begin{aligned} x \cdot x &= (x \cdot x) + 0 && \text{elemento neutro} \\ &= (x \cdot x) + (x \cdot x') && \text{complemento} \\ &= x \cdot (x + x') && \text{distributiva} \\ &= x \cdot 1 && \text{complemento} \\ &= x && \text{elemento neutro} \end{aligned}$$

L'idempotenza di $+$ ($x + x = x$) segue per dualità.

1.2 Algebra di commutazione

Definizione. [Algebra di commutazione (*switching algebra*)] È l'algebra di Boole il cui supporto è costituito dai soli valori $\{0, 1\}$:

$$\mathcal{B} = \langle \{0, 1\}, +, \cdot, ', 0, 1 \rangle.$$

Ogni variabile può assumere solo i valori 0 e 1.

È l'algebra usata da Shannon per l'analisi dei circuiti. I tre operatori sono definiti dalle seguenti tabelle di verità.

$+$	0	1
0	0	1
1	1	1

$\cdot$	0	1
0	0	0
1	0	1

x	x'
0	1
1	0

1.2.1 OR esclusivo

L'*OR esclusivo* (XOR) è l'operatore $\oplus$ definito da

$$a \oplus b = a'b + ab',$$

cioè vale 1 quando a e b sono diversi.

$\oplus$	0	1
0	0	1
1	1	0

Dalla definizione, applicando le proprietà dell'algebra di Boole, si ricava:

$$x \oplus 1 = x' \qquad x \oplus 0 = x \tag{1}$$

$$x \oplus x = 0 \qquad x \oplus x' = 1 \tag{2}$$

$$x' \oplus y = x \oplus y' = (x \oplus y)' \tag{3}$$

Esercizi per casa

Esercizio. Usare i diagrammi di Venn per dimostrare le proprietà (associatività, idempotenza, ...) dell'algebra booleana su supporto finito.

Esercizio. Dimostrare che l'algebra di commutazione è un'algebra di Boole, cioè verificare i postulati di Huntington.

Esercizio. Dimostrare, per l'algebra di commutazione, associatività, idempotenza, elemento nullo, unicità del complemento, assorbimento, semplificazione, involuzione, De Morgan e consenso.

2 Funzioni ed espressioni booleane

Lezione 2 – 6 ottobre 2026

2.1 Funzioni booleane

Definizione. [Funzione booleana] Sia $\mathcal{B} = \langle A = \{0,1\}, +, \cdot, ', 0, 1\rangle$ un'algebra di commutazione. Una *funzione booleana* a n variabili è una relazione

$$f\colon A^n \to A$$

che mette in corrispondenza gli elementi del dominio A^n con gli elementi del codominio A.

Dal punto di vista operativo una funzione booleana si definisce con le quattro regole seguenti.

1. $\forall a \in A$, la *funzione costante* $f(x_1, \ldots, x_n) = a$ è una funzione booleana a n variabili.
2. $\forall x_i \in \{x_1, \ldots, x_n\}$, la *funzione proiezione* $f(x_1, \ldots, x_n) = x_i$ è una funzione booleana a n variabili.
3. Se g e h sono funzioni booleane a n variabili, lo sono anche $g + h$, $g \cdot h$ e g', definite da
$$\begin{aligned}(g+h)(x_1,\ldots,x_n) &= g(x_1,\ldots,x_n) + h(x_1,\ldots,x_n)\\ (g\cdot h)(x_1,\ldots,x_n) &= g(x_1,\ldots,x_n)\cdot h(x_1,\ldots,x_n)\\ (g')(x_1,\ldots,x_n) &= \big(g(x_1,\ldots,x_n)\big)'\end{aligned}$$
4. Non esistono altre funzioni booleane oltre a quelle ottenute dall'applicazione iterativa delle regole precedenti.

2.1.1 Tabelle di verità

In un'algebra di commutazione $A = \{0,1\}$, quindi le funzioni booleane hanno la forma

$$f\colon \{0,1\}^n \to \{0,1\}.$$

Il dominio di f ha 2^n elementi e il codominio ne ha 2. La funzione può essere descritta dalla sua *tabella di verità* T_f, che contiene una riga per ciascuna configurazione delle n variabili in ingresso, con il relativo valore di uscita di f.

Esempio. Funzione booleana sulle variabili x, y, z che vale 1 se e solo se *esattamente due* variabili valgono 1.

x	y	z	$f(x,y,z)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Proposizione. Il numero di funzioni booleane (su un'algebra di commutazione) di n variabili è 2^{2^n}.

Infatti, date n variabili si hanno 2^n configurazioni (tante quanti gli elementi di $\{0,1\}^n$) e ogni funzione è definita dall'insieme delle configurazioni per cui assume valore 1. Il numero di funzioni è quindi la cardinalità dell'insieme delle parti di $\{0,1\}^n$, cioè 2^{2^n}.

2.1.2 Esempi

Esempio (Funzioni di una variabile)**.** Alcune di queste hanno particolare importanza dal punto di vista della realizzazione.

x	$f(x)$	$f(x)$	$f(x)$	$f(x)$
0	0	0	1	1
1	0	1	0	1
	costante 0	proiezione di x	negazione di x	costante 1

Esempio (Funzioni di due variabili (I))**.**

x	y	$f(x,y)$					
0	0	0	1	0	0	1	1
0	1	0	1	0	1	1	0
1	0	0	1	1	0	0	1
1	1	0	1	1	1	0	0
		costante 0	costante 1	proiez. x	proiez. y	negaz. x	negaz. y

Esempio (Funzioni di due variabili (II))**.**

x	y	$f(x,y)$					
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1
		AND	OR	NAND	NOR	XOR	EQUIV.

2.1.3 Funzioni non completamente specificate

Talvolta una funzione f viene specificata solo in parte e si dice *non completamente specificata*. Ci sono due motivi principali:

1. la natura del problema garantisce che una certa combinazione delle variabili di ingresso non possa presentarsi;
2. non ha importanza quale valore assuma la funzione per una certa combinazione delle variabili di ingresso.

Nel primo caso si dice che la funzione *non è controllabile* in corrispondenza di quella combinazione di ingressi, nel secondo che *non è osservabile*.

Esempio. Sia f una funzione di due variabili x_1, x_2 tale che

$$\big(f(x_1,x_2)=1 \iff x_1=x_2=0\big) \wedge \big(f(x_1,x_2)=0 \iff x_1 \neq x_2\big).$$

Il valore di $f(1,1)$ non è specificato (nella tabella si indica con $-$).

x_1	x_2	$f(x_1,x_2)$
0	0	1
0	1	0
1	0	0
1	1	$-$

2.2 Espressioni booleane

Definizione. [Espressione booleana] Un'*espressione booleana* definita su un insieme di variabili booleane $X = \{x_1, x_2, \ldots, x_n\}$ è definita dalle regole seguenti:

- gli elementi del supporto sono espressioni;
- le variabili $x_1, x_2, \ldots, x_n$ sono espressioni;
- se h, g sono espressioni booleane, lo sono anche $h + g$, $h \cdot g$ e h';
- non esistono altre espressioni oltre a quelle ottenute applicando iterativamente le regole precedenti.

2.2.1 Notazioni e convenzioni

Fissato l'insieme dei simboli usati per le variabili, si può eliminare il simbolo di prodotto logico $\cdot$ nei termini prodotto. Per le proprietà dell'algebra booleana si possono inoltre omettere alcune parentesi, ricordando che il prodotto logico ha *priorità* sulla somma logica.

Esempio.

- $x + (y \cdot z)$ si scrive $x + yz$ (si elimina il simbolo $\cdot$ nel termine prodotto $y \cdot z$ e si usa la priorità di $\cdot$ rispetto a $+$);
- per la proprietà associativa, $x + (y + z \cdot x)$ si scrive $x + y + zx$ (si eliminano le parentesi per l'associatività e il simbolo $\cdot$ nel termine prodotto $z \cdot x$).

2.2.2 Letterali

Un *letterale* è un'occorrenza di una variabile (in forma naturale o negata) all'interno di un'espressione.

Esempio. L'espressione $(x' + y') \cdot (x + x')$ contiene 4 letterali:

- un'occorrenza della variabile x in forma naturale;
- due occorrenze della variabile x in forma negata (x');
- un'occorrenza della variabile y in forma negata (y').

2.2.3 Valutazione di un'espressione

Sia e un'espressione booleana sulle variabili $x_1, \ldots, x_n$ e sia $(v_1, \ldots, v_n) \in \{0,1\}^n$ una configurazione delle variabili. La *valutazione* di e rispetto a $(v_1, \ldots, v_n)$ è il procedimento con cui si sostituisce a ciascuna variabile il suo valore e, applicando le regole dell'algebra di commutazione, si ottiene un valore $v \in \{0,1\}$ per l'espressione. Ripetendo il procedimento per tutte le configurazioni si ottiene una tabella di verità: *ad ogni espressione corrisponde dunque una funzione.*

Esempio. Sia $e = z(y + x) + xy$. La valutazione per $(1,0,0)$ è

$$e(100) = 0 \cdot (0 + 1) + 1 \cdot 0 = 0.$$

La valutazione completa determina la tabella di verità della funzione booleana su x, y, z:

x	y	z	e
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

2.2.4 Espressioni e funzioni

Ad una singola espressione corrisponde una singola funzione; ad una funzione possono invece corrispondere molte espressioni.

Definizione. [Espressioni equivalenti] Due espressioni e_1, e_2 definite sullo stesso insieme di variabili si dicono *equivalenti*, e si scrive $e_1 \equiv e_2$, se corrispondono alla stessa funzione.

Le trasformazioni eseguite su un'espressione di partenza e_0 usando i postulati e le proprietà dell'algebra booleana portano a espressioni equivalenti.

2.2.5 Espressioni a confronto

Nei contesti in cui si usa un'algebra booleana è fondamentale rappresentare le funzioni con *buone* espressioni. Un possibile criterio di bontà consiste nel minimizzare il numero di letterali: si individua una prima espressione che rappresenti la funzione e si cerca poi un'espressione equivalente migliore (con meno letterali). Espressioni di partenza tipiche da manipolare per minimizzare i letterali sono le espressioni in *forma canonica*. Le forme canoniche di base sono due: la prima espressa come somma di prodotti (SOP), la seconda come prodotto di somme (POS); entrambe si ottengono dal teorema di espansione di Shannon.

2.3 Forme canoniche

2.3.1 Teorema di espansione di Shannon

Teorema (Espansione di Shannon). Sia $f\colon B^n \to B$ una funzione booleana di n variabili. Allora per ogni $(x_1, \ldots, x_n) \in B^n$

$$f(x_1, \ldots, x_n) = x_1' \cdot f(0, x_2, \ldots, x_n) + x_1 \cdot f(1, x_2, \ldots, x_n)$$

e dualmente

$$f(x_1, \ldots, x_n) = \big(x_1' + f(1, x_2, \ldots, x_n)\big) \cdot \big(x_1 + f(0, x_2, \ldots, x_n)\big).$$

I termini $f(0, x_2, \ldots, x_n)$ e $f(1, x_2, \ldots, x_n)$ si chiamano *cofattori* e si indicano spesso con $f_{x_1'}$ e f_{x_1}.

Esempio (Applicazione iterata, forma somma)**.** Per una funzione di due variabili $f(x, y)$:

$$\begin{aligned} f(x, y) &= x' \cdot f(0, y) + x \cdot f(1, y) \\ &= x'\big(y' \cdot f(0, 0) + y \cdot f(0, 1)\big) + x\big(y' \cdot f(1, 0) + y \cdot f(1, 1)\big) \\ &= x'y' \cdot f(00) + x'y \cdot f(01) + xy' \cdot f(10) + xy \cdot f(11). \end{aligned}$$

In generale, l'applicazione iterata del teorema a una funzione di n variabili permette di riscriverla come somma di 2^n termini:

$$\begin{aligned} f(x_1,\dots,x_n) = {} & x_1'\cdots x_{n-1}'x_n' \cdot f(0\dots00) \\ & + x_1'\cdots x_{n-1}'x_n \cdot f(0\dots01) \\ & + \cdots \\ & + x_1\cdots x_{n-1}x_n \cdot f(1\dots11). \end{aligned}$$

Esempio (Applicazione iterata, forma prodotto)**.** Applicando la forma duale a $f(x,y)$:

$$\begin{aligned} f(x,y) &= \big(x' + f(1,y)\big)\big(x + f(0,y)\big) \\ &= \big(x' + [(y' + f(11))(y + f(10))]\big)\big(x + [(y' + f(01))(y + f(00))]\big) \\ &= (x' + y' + f(11))(x' + y + f(10))(x + y' + f(01))(x + y + f(00)). \end{aligned}$$

In generale si ottiene il prodotto di 2^n termini:

$$\begin{aligned} f(x_1,\dots,x_n) = {} & (x_1 + \cdots + x_{n-1} + x_n + f(0\dots00)) \\ & \cdot(x_1 + \cdots + x_{n-1} + x_n' + f(0\dots01)) \\ & \cdot\cdots \\ & \cdot(x_1' + \cdots + x_{n-1}' + x_n' + f(1\dots11)). \end{aligned}$$

2.3.2 Prima forma canonica: mintermini

Definizione. [Mintermine] Un *mintermine* su $x_1,\dots,x_n$ è un termine prodotto $y_1\cdot y_2\cdots y_n$ dove $y_i = x_i$ oppure $y_i = x_i'$, per ogni $i = 1,\dots,n$.
Dato $\bar{v} = (v_1,\dots,v_n) \in \{0,1\}^n$ si costruisce il mintermine $m_{\bar{v}} = y_1 \cdot y_2 \cdots y_n$ con $y_i = x_i$ se $v_i = 1$ e $y_i = x_i'$ se $v_i = 0$ (il mintermine *corrispondente* alla configurazione $\bar{v}$).

Il mintermine $m_{\bar{v}}$ è un'espressione che vale 1 in corrispondenza dell'*unica* configurazione di ingresso $\bar{v}$.

Esempio. Sia $\bar{v} = (101)$ una configurazione per le variabili $\{x,y,z\}$. Il mintermine associato è $m_{\bar{v}} = xy'z$, che assume valore 1 soltanto se $x = 1$, $y = 0$ e $z = 1$, cioè per la sola configurazione (101).

Definizione. [Prima forma canonica (*Sum of Products*, SOP)] Data una funzione booleana $f(x_1,\dots,x_n)$, la sua rappresentazione in *prima forma canonica* è la somma di tutti i mintermini su $x_1,\dots,x_n$ associati alle configurazioni per cui f vale 1.

L'applicazione iterata del teorema di Shannon porta alla prima forma canonica. In pratica, per ricavarla basta guardare le righe della tabella di verità dove f vale 1.

Esempio. Sia f la funzione su x,y,z che vale 1 se e solo se esattamente due variabili valgono 1 (tabella dell'esempio all'inizio della lezione). I mintermini associati alle configurazioni per cui $f = 1$ sono:

- $x'yz$, dalla quarta riga (011);
- $xy'z$, dalla sesta riga (101);
- xyz', dalla settima riga (110).

Quindi la prima forma canonica di f è

$$f = x'yz + xy'z + xyz'.$$

2.3.3 Seconda forma canonica: maxtermini

Definizione. [Maxtermine] Un *maxtermine* su $x_1, \ldots, x_n$ è un termine somma $y_1 + y_2 + \cdots + y_n$ dove $y_i = x_i$ oppure $y_i = x_i'$, per ogni $i = 1, \ldots, n$.
Data la configurazione $\bar{v} = (v_1, \ldots, v_n) \in \{0,1\}^n$ si costruisce il maxtermine $M_{\bar{v}} = y_1 + y_2 + \cdots + y_n$ con $y_i = x_i$ se $v_i = 0$ e $y_i = x_i'$ se $v_i = 1$ (il maxtermine *corrispondente* alla configurazione $\bar{v}$).

Il maxtermine $M_{\bar{v}}$ è un'espressione che vale 0 in corrispondenza dell'*unica* configurazione di ingresso $\bar{v}$.

Esempio. Per $\bar{v} = (101)$ su $\{x, y, z\}$ il maxtermine associato è $M_{\bar{v}} = x' + y + z'$, che assume valore 0 soltanto se $x = 1$, $y = 0$ e $z = 1$.

Definizione. [Seconda forma canonica (*Product of Sums*, POS)] Data una funzione booleana $f(x_1, \ldots, x_n)$, la sua rappresentazione in *seconda forma canonica* è il prodotto di tutti i maxtermini su $x_1, \ldots, x_n$ corrispondenti alle configurazioni per cui f vale 0.

L'applicazione iterata del teorema di Shannon nella sua forma duale porta alla seconda forma canonica. In pratica, per ricavarla basta guardare le righe della tabella di verità dove f vale 0.

Esempio. Per la stessa funzione (esattamente due variabili valgono 1) i maxtermini associati alle configurazioni per cui $f = 0$ sono:

- $x + y + z$, dalla prima riga (000);
- $x + y + z'$, dalla seconda riga (001);
- $x + y' + z$, dalla terza riga (010);
- $x' + y + z$, dalla quinta riga (100);
- $x' + y' + z'$, dall'ultima riga (111).

Quindi la seconda forma canonica di f è

$$f = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)(x' + y' + z').$$

Riferimenti

Capitolo 2 del libro di testo: C. Bolchini et al., *Reti Logiche*, Maggioli Editore. Lezione 2: sezione 2.3; lezione 3: sezioni 2.4 e 2.5.

Esercizi per casa

Esercizio. Definire la tabella di verità della funzione booleana $f\colon B^3 \to B$ che assume valore 1 soltanto se almeno due bit in ingresso hanno valore 0.

Esercizio. Rappresentare f mediante un'espressione booleana in prima forma canonica.

Esercizio. Rappresentare f' mediante un'espressione booleana in seconda forma canonica.

3 Porte logiche e circuiti logici

Lezione 3

3.1 Porte logiche

Definizione. [Porta logica] Una *porta logica* è un componente elettronico di base che opera su uno o più segnali di ingresso e genera un unico segnale in uscita. È la realizzazione fisica delle funzioni elementari introdotte dall'algebra (AND, OR, NOT).

Oltre alle porte AND, OR e NOT, si usano le porte NAND, NOR, XOR e XNOR.

x_0	x_1	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

La porta NOT ha un solo ingresso e restituisce x'.

3.1.1 NAND e NOR: operatori funzionalmente completi

La porta NAND realizza $(x_0 \cdot x_1)'$ e la porta NOR realizza $(x_0 + x_1)'$.

Proposizione. NAND e NOR sono *funzionalmente completi*: qualsiasi funzione logica si può realizzare usando esclusivamente porte NAND, oppure esclusivamente porte NOR, invece di AND, OR e NOT.

Esempio (NOT con una NAND)**.** Collegando i due ingressi di una NAND alla stessa variabile x si ottiene, per idempotenza,

$$(x \cdot x)' = (x)' = x'.$$

Quindi la porta NOT si realizza con una NAND.

3.1.2 XOR e XNOR

Le porte XOR e XNOR corrispondono agli operatori booleani XOR e XNOR (OR e NOR *esclusivi*): realizzano rispettivamente la funzione di *disuguaglianza* e di *uguaglianza* tra due bit. Lo XOR vale 1 se e solo se i due ingressi sono diversi, lo XNOR vale 1 se e solo se sono uguali.

3.1.3 Porte a n ingressi

Accanto alle porte elementari a due ingressi, in genere si assume di avere a disposizione le stesse porte con un numero arbitrario di ingressi. In una AND a tre ingressi x_0, x_1, x_2 l'uscita y vale 1 se e solo se tutte le variabili in ingresso valgono 1. In una OR a quattro ingressi $x_0, \ldots, x_3$ l'uscita y vale 1 se almeno una variabile in ingresso vale 1.

3.2 Circuiti logici

Le porte elementari si collegano per realizzare funzioni più complesse. A partire da un'espressione logica e (ricavata, per esempio, dalla tabella di verità di una funzione booleana) si costruisce il circuito che realizza la funzione descritta da e.

Esempio. Il circuito per $x + y'z$ usa una porta NOT su y, una AND tra y' e z (uscita $y'z$) e una OR tra x e $y'z$.

3.2.1 Ritardi e cammino critico

Ogni porta logica ha un *ritardo*: il tempo tra l'istante in cui si applicano i segnali in ingresso e l'istante in cui il segnale di uscita si stabilizza a un valore costante. Il ritardo dipende dalla realtà fisica dei componenti (i transistor).

> **Definizione.** [Cammino critico] Il *cammino critico* (*critical path*) è il cammino dagli ingressi alle uscite che attraversa il maggior numero di porte in un circuito, trascurando i negatori, perché possono essere accorpati alla porta precedente o successiva.

Esempio. In una rete divisa in tre livelli di porte, i cammini critici attraversano tre porte.

3.2.2 Circuiti da forme canoniche

Le funzioni espresse in prima o in seconda forma canonica corrispondono sempre a circuiti logici *a due livelli*, che usano porte a n ingressi.

- In *prima forma canonica* il primo livello contiene le porte AND che calcolano i mintermini e il secondo livello un'unica porta OR che ne fa la somma.
- In *seconda forma canonica* il primo livello contiene le porte OR che calcolano i maxtermini e il secondo livello un'unica porta AND che ne fa il prodotto.

Esempio. Sia $f(x, y, z)$ la funzione che vale 1 solo per 011, 101, 110:

x	y	z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

La prima forma canonica è $f = x'yz + xy'z + xyz'$. Il circuito ha tre NOT per le variabili negate, tre AND a tre ingressi (una per mintermine) e una OR a tre ingressi.

3.3 Circuiti speciali: i sommatori

Con le porte logiche si costruiscono circuiti che eseguono operazioni aritmetiche. Vediamo tre sommatori.

3.3.1 Half adder

Un *half adder* (HA) ha come ingressi due bit a, b e come uscite due bit: s (la somma) e c (il riporto, *carry*).

a	b	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

La colonna s è uno XOR, la colonna c è un AND:

$$s = a \oplus b, \qquad c = a \cdot b.$$

Il circuito è quindi formato da una porta XOR e una porta AND con gli stessi ingressi.

3.3.2 Full adder

Un *full adder* (FA) ha tre ingressi, a, b e c_{in} (bit e riporto in ingresso), e due uscite, s e c_{out} (somma e riporto in uscita).

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Poiché la somma è associativa, $a + b + c_{in} = (a + b) + c_{in}$: si possono usare due half adder, il primo per $a + b$ e il secondo per sommare c_{in} al risultato, e combinare i due riporti con una OR per ottenere c_{out}.

3.3.3 Sommatore a n bit (ripple-carry)

Un *sommatore a n bit* ha come ingressi $2n$ bit, $a_{n-1} \dots a_0$ e $b_{n-1} \dots b_0$, e come uscite n bit $s_{n-1} \dots s_0$ più il riporto finale c, tali che la sequenza in uscita rappresenti in binario la somma dei due numeri in ingresso. Si realizza collegando n full adder in cascata: il riporto in uscita di ciascuno entra come riporto in ingresso del successivo (da cui il nome *ripple-carry*).

Esempio. Per sommare due numeri a 3 bit $X = x_2x_1x_0$ e $Y = y_2y_1y_0$ servono 3 full adder in cascata. Ad esempio $100 + 110 = 1010$.

Riferimenti. Libro di testo (C. Bolchini et al., *Reti Logiche*, Maggioli): cap. 2, sottosezioni 2.6 (porte logiche) e 2.7 (circuiti logici); cap. 5, sottosezione 5.2.1 (sommatori ripple-carry). Esercizi 2.24, 2.25, 2.26, 2.27.

4 Progettazione di reti logiche combinatorie

Lezione 4 Una rete logica *combinatoria* ha un comportamento che dipende solo dagli ingressi, perché è priva di memoria. Vediamo come si progetta e si ottimizza.

4.1 Le fasi della progettazione

Progettare un circuito combinatorio comporta quattro passi successivi.

1. **Comprensione e analisi della specifica semi-formale.** La descrizione iniziale di un problema spesso non è completa e va riscritta per eliminare ogni ambiguità. Casi tipici: gestione delle condizioni limite o dei casi di errore, analisi dell'ambiente in cui la rete opera, scelta della codifica delle informazioni. Alla fine si ha ancora una specifica semi-formale, ma completa.

2. **Formalizzazione della specifica.** Si traduce la specifica in un linguaggio formale, più adatto alla manipolazione: le *tabelle di verità*, che specificano la relazione richiesta tra ingressi e uscite.
3. **Sintesi.** Si passa dalle tabelle di verità a espressioni algebriche booleane. In questa fase si trascura ogni considerazione sulla qualità.
4. **Ottimizzazione.** Fissato un criterio di qualità, si manipola l'espressione ottenuta nella sintesi per avere un'espressione funzionalmente equivalente ma migliore.

Le caratteristiche che misurano la qualità di una rete sono generalmente l'area della realizzazione fisica, il tempo necessario a produrre un risultato, la potenza e l'energia assorbita (*metriche d'area*, *metriche di tempo*, ...). Alcune metriche d'area comuni sono il numero di porte logiche generiche, il numero di porte a due ingressi, il numero di implicanti/implicati e il numero di letterali.

4.2 Esempio: riconoscere pari e dispari

Si progetti una rete combinatoria in grado di discriminare i numeri pari dai numeri dispari.

Fase 1: specifica

- Ci interessano i numeri interi e dobbiamo fissare l'intervallo, in base all'applicazione. Scegliamo per semplicità i giorni della settimana, cioè l'intervallo $[0, 6]$.
- Servono $\lceil \log_2 7 \rceil = 3$ bit per rappresentarlo.
- L'uscita è binaria (pari o dispari), quindi basta un bit. Fissiamo la codifica 1 = pari e 0 = dispari.
- Con l'intervallo $[0, 6]$ l'ingresso 111 (il numero 7) non è valido. Introduciamo un secondo bit di uscita per segnalare l'errore.

La specifica semi-formale completa è: *una rete combinatoria con un ingresso a tre bit* $X = \{x_0, x_1, x_2\}$ *che rappresenta un intero in* $[0, 6]$ *in codifica binaria naturale e due uscite di un bit ciascuna:* z *indica se il valore è pari* ($z = 1$) *o dispari* ($z = 0$)*, e indica un errore* ($e = 1$) *o il funzionamento corretto* ($e = 0$)*.*

Fase 2: formalizzazione

La rete realizza le funzioni $z(x_0, x_1, x_2)$ ed $e(x_0, x_1, x_2)$, con x_0 bit più significativo:

x_0	x_1	x_2	z	e
0	0	0	1	0
0	0	1	0	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	0
1	1	1	0	1

Fase 3: sintesi

Dalle tabelle si ricavano le espressioni in prima forma canonica:

$$z = x_0' x_1' x_2' + x_0' x_1 x_2' + x_0 x_1' x_2' + x_0 x_1 x_2',$$
$$e = x_0 x_1 x_2.$$

Da qui si potrebbe già derivare un circuito, ma le proprietà dell'algebra booleana permettono di ottimizzare (in numero di letterali).

Fase 4: ottimizzazione

Raccogliendo x_2' e usando $y + y' = 1$ due volte:

$$z = x_2' \left(x_0' x_1' + x_0' x_1 + x_0 x_1' + x_0 x_1\right) = x_2' \left(x_0'(x_1' + x_1) + x_0(x_1' + x_1)\right) = x_2'.$$

Quindi $z = x_2'$: un numero è pari se e solo se l'ultimo bit è 0. Le tecniche sistematiche di ottimizzazione sono oggetto delle lezioni successive (mappe di Karnaugh).

4.3 Secondo esempio: display a sette segmenti

Un display a 7 segmenti è costituito da 7 segmenti luminosi che si accendono o spengono a formare il simbolo di una cifra; per controllarlo servono 7 segnali binari. Si progetti una rete combinatoria che gestisca un display a sette segmenti di una cifra decimale nell'intervallo $[0, 7]$ in codifica binaria naturale: la rete ha 3 ingressi e 7 uscite, una per segmento.

Riferimenti. Libro di testo (C. Bolchini et al., *Reti Logiche*, Maggioli): cap. 4, sottosezioni 4.1, 4.2, 4.3. Esercizi per casa: 4.1, 4.2, 4.5, 4.6, 4.7, 4.9.

5 Ottimizzazione: mappe di Karnaugh

Lezione 5

5.1 Che cos'è una mappa di Karnaugh

Le mappe di Karnaugh sono un metodo *grafico* per semplificare le espressioni booleane. Furono inventate nel 1953 da Maurice Karnaugh, ingegnere delle telecomunicazioni dei Bell Labs, e sono molto efficaci per ottimizzare reti combinatorie con poche variabili (fino a quattro).

> **Definizione.** [Mappa di Karnaugh] Una *mappa di Karnaugh* è una rappresentazione formale, alternativa alla tabella di verità, di una funzione booleana: una griglia con una cella per ogni configurazione delle variabili, in cui si scrive il valore della funzione.

Esempio. Sia $f(x, y, z)$ con $f(001) = f(011) = 1$ e $f = 0$ altrove. Le righe della mappa indicano x e le colonne indicano yz:

	00	01	11	10
0	0	1	1	0
1	0	0	0	0

Le due celle con 1 sono quelle di $f(001)$ e $f(011)$.

5.2 Codice di Gray

Nella mappa le configurazioni delle variabili non seguono l'ordine binario naturale $(00, 01, 10, 11)$ ma un *codice di Gray* (o codice a specchio): $00, 01, 11, 10$. Passando da una colonna alla successiva cambia un solo bit. Ciò è fondamentale perché mette in celle *adiacenti* (con un lato in comune) i mintermini che danno luogo a semplificazioni.

Per esempio, nella mappa precedente

$$x'y'z + x'yz = x'z\,(y' + y) = x'z.$$

5.3 Perché funzionano

Due mintermini sono *logicamente adiacenti* se sono uguali a meno di una variabile, che compare in forma naturale in uno e negata nell'altro (per esempio $x_1x_2'x_3$ e $x_1x_2x_3$). Se $p_1 + p_2$ è la somma di due mintermini logicamente adiacenti, allora $p_1 + p_2$ si può semplificare:

$$x_1x_2'x_3 + x_1x_2x_3 = x_1x_3\,(x_2' + x_2) = x_1x_3.$$

In una mappa di Karnaugh, a celle geometricamente adiacenti corrispondono mintermini logicamente adiacenti.

5.4 Come si usa

1. Si raggruppano gli 1 che compaiono in celle adiacenti: ogni gruppo è un *k-cubo*.
2. Da ogni k-cubo si ricava un *implicante*, cioè un termine prodotto p tale che, se p vale 1, allora f vale 1. L'implicante è il prodotto delle variabili che compaiono sempre in forma naturale (oppure sempre in forma negata) in tutte le celle del k-cubo.
3. Coperti tutti gli 1, si sommano gli implicanti: si ottiene un'espressione ottimizzata, in forma di somma di prodotti, corrispondente a f.

Regole per generare i k-cubi

1. La mappa si immagina *arrotolata* su se stessa: l'ultima colonna è adiacente alla prima e l'ultima riga è adiacente alla prima.
2. I lati dei rettangoli in cui si raccolgono gli 1 devono essere potenze di 2.
3. Ogni k-cubo deve essere il più grande possibile.
4. Ogni 1 deve stare in almeno un k-cubo.
5. Un 1 può stare in più k-cubi, se serve per coprire tutti gli 1.

5.5 Esempi

Esempio (Tre variabili)**.** Mappa con righe x e colonne yz e $f = y'z' + yz$:

	00	01	11	10
0	1	0	1	0
1	1	0	1	0

I due k-cubi sono la colonna 00 (implicante $y'z'$) e la colonna 11 (implicante yz).

Esempio (Tre variabili, con la colonna 10)**.**

	00	01	11	10
0	0	1	1	1
1	1	1	0	1

Si hanno tre k-cubi: le celle $01, 11$ della riga 0 (implicante $x'z$), le celle $00, 01$ della riga 1 (implicante xy') e la colonna 10 (implicante yz'). Quindi

$$f(x, y, z) = x'z + xy' + yz'.$$

Esempio (Quattro variabili)**.** Righe xy, colonne zw:

	00	01	11	10
00	0	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	0

I k-cubi sono: le righe 11 e 10 nelle colonne $00, 01$ (implicante xz'); le celle 01 e 11 della colonna 01 (implicante $yz'w$); le celle 00 e 01 della colonna 10 (implicante $x'zw'$). Quindi

$$f(x, y, z, w) = xz' + yz'w + x'zw'.$$

Esempio (Quattro variabili, con celle ai bordi)**.**

	00	01	11	10
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	0

Rispetto all'esempio precedente la cella $(00, 00)$ vale 1. Va raccolta con la cella $(10, 00)$, perché la prima e l'ultima riga sono adiacenti: si ottiene l'implicante $y'z'w'$. Quindi

$$f(x, y, z, w) = xz' + yz'w + x'zw' + y'z'w'.$$

Esempio (Quattro variabili, cubo grande)**.**

	00	01	11	10
00	1	0	1	1
01	0	0	1	1
11	0	0	1	1
10	1	0	1	1

Le colonne 11 e 10 formano un k-cubo 4×2 (implicante z). Le quattro celle d'angolo (righe $00, 10$ e colonne $00, 10$) formano un secondo k-cubo, perché la mappa è arrotolata sia in orizzontale sia in verticale (implicante $y'w'$). Quindi

$$f(x, y, z, w) = z + y'w'.$$